GUIA PRÁTICO · PARA QUEM TRABALHA COM TI

Claude × Claude Code



Mesma inteligência.
Outro nível de trabalho.

As principais diferenças entre o Claude do chat e o Claude Code, por que o Claude Code rende muito mais no dia a dia de TI — e como os plugins multiplicam esse ganho.

~/projetos/api-pedidos — claude

```
~/projetos/api-pedidos $ claude

> A API está retornando erro 500 desde o último deploy. Investigue e corrija.

● Lendo logs/app.log ····· KeyError: 'DB_URL' · 37 ocorrências
● git diff v2.3.0..HEAD ····· variável renomeada para DATABASE_URL
● Editando config/settings.py ····· +2 -1
● pytest ····· 148 passed
● git commit ····· fix(config): usa DATABASE_URL na carga da config

Pronto. O deploy renomeou a variável só no código. Quer que eu abra o pull request?
```

Sessão ilustrativa do Claude Code no terminal

01 As principais diferenças

02 Por que o Claude Code rende mais

03 Plugins e suas vantagens

Outubro de 2026

Baseado na documentação oficial: code.claude.com e claude.com

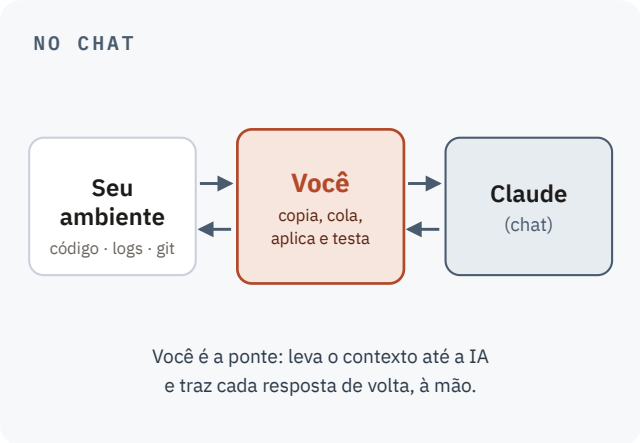
ANTES DE TUDO

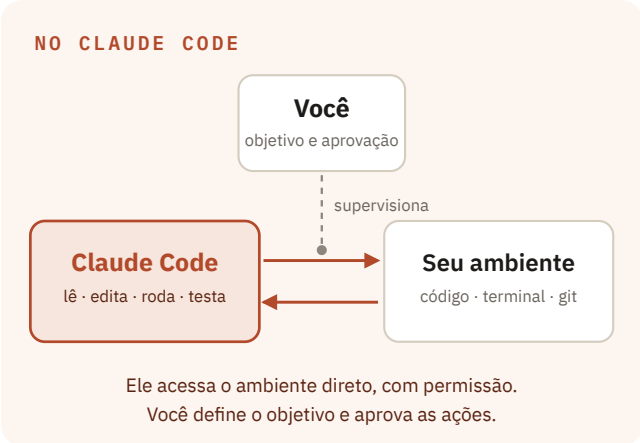
O chat conversa sobre o seu trabalho. O Claude Code faz o trabalho — no seu ambiente, com a sua permissão.

Mesmo cérebro, mãos diferentes

Os dois usam os mesmos modelos Claude. A diferença não está na inteligência, e sim no **alcance** dela. No chat, o Claude responde com texto — e, quando executa código, faz isso num contêiner isolado na nuvem, separado dos seus sistemas. Quem abre os arquivos, copia, cola, executa e testa é você.

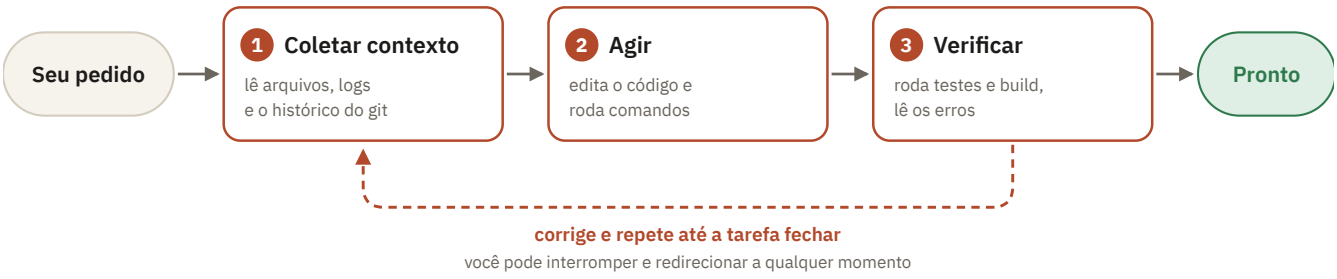
O Claude Code coloca o mesmo modelo dentro do que a documentação chama de *harness* agêntico: ferramentas para ler e editar arquivos, pesquisar o código, rodar comandos no terminal, usar o git e consultar a web. Com elas, ele trabalha em ciclo até a tarefa ficar pronta — e você acompanha e aprova.





O ciclo que faz a diferença

Quando você passa uma tarefa, o Claude Code trabalha em três fases que se repetem: **coletar contexto**, **agir** e **verificar o resultado**. Cada ação gera informação nova, que decide o próximo passo — ele encadeia dezenas de ações e se corrige pelo caminho.



> *"Embora seja excelente em código, ele ajuda em qualquer coisa que se faça pela linha de comando: escrever documentação, rodar builds, pesquisar arquivos, investigar assuntos e muito mais."*
>
> Documentação oficial do Claude Code — "How Claude Code works" (tradução livre)

PARTE 1

As principais diferenças

Lado a lado, o que muda quando o Claude sai da janela de conversa e passa a trabalhar dentro do seu ambiente.

Aspecto	CHAT Claude (claude.ai)	AGENTE Claude Code
Onde você usa	Navegador, app desktop e celular	Terminal, VS Code e Cursor, JetBrains, aba **Code** do app desktop, navegador (claude.ai/code), Slack e pipelines de CI/CD
O que ele enxerga	Só o que você cola ou anexa na conversa	O projeto inteiro: abre, pesquisa e relaciona os arquivos sozinho
Onde as coisas rodam	Num contêiner isolado na nuvem, sem acesso às suas máquinas	No seu ambiente — seu terminal, suas ferramentas, seu git — ou em VMs na nuvem, se você preferir
Como as mudanças chegam	Ele escreve o código; você copia e aplica	Ele edita os arquivos direto, vários de uma vez, e mostra o diff para você revisar
Validação	Você executa e traz o erro de volta para a conversa	Ele roda testes e build, lê a saída e corrige até passar
Git	Fica por sua conta	Cria branch, faz commit com mensagem descritiva e abre pull request
Memória	Projetos e memória entre conversas	`CLAUDE.md` versionado no repositório + memória automática por projeto
Integrações	Conectores remotos (MCP)	MCP local e remoto: bancos de dados, Jira, Slack, ferramentas internas
Automação	Uso dentro do app; não entra em scripts nem pipelines	Modo não interativo (`claude -p`), pipes, hooks, GitHub Actions, GitLab CI/CD e rotinas agendadas
Desfazer	Não se aplica: ele não altera seus arquivos	Checkpoints: toda edição de arquivo pode ser revertida (`Esc` duas vezes)
Controle	Isolado: não toca nos seus sistemas	Modos de permissão, regras allow/ask/deny por comando, sandbox e políticas da empresa
Plugins	Carrega só parte: skills, comandos e conectores remotos	Carrega tudo: skills, subagentes, hooks, MCP local, LSP, executáveis...
Planos	Free, Pro, Max, Team e Enterprise	Incluído no Pro, Max, Team e Enterprise (não está no Free)

Mesma conta, sem assinatura extra. Quem assina o Pro ou o Max já tem o Claude Code incluído: basta instalar e entrar com a mesma conta. No app desktop do Claude, ele já vem pronto na aba **Code**, sem instalar nada no terminal.

PARTE 1 · NA PRÁTICA

O mesmo problema, dois mundos

Situação clássica: depois do último deploy, a API começou a responder erro 500. Veja como a investigação acontece em cada ferramenta.

NO CHAT

Você é o intermediário

1. Você descreve o problema e cola o trecho de log que parece relevante.
2. O Claude levanta hipóteses e pede mais dados: a configuração, a versão, o código da rota.
3. Você abre os arquivos, copia e cola na conversa — torcendo para não ter esquecido nenhum.
4. Recebe uma correção e aplica à mão, arquivo por arquivo.
5. Roda os testes. Algo quebra. Copia o erro e volta ao passo 2.

Cada volta do ciclo passa por você: olhos, mãos e Ctrl+C / Ctrl+V.

NO CLAUDE CODE

Você é o revisor

```
$ claude "A API retorna 500 desde o último deploy. Investigue e corrija."
```

1. Lê os logs e o `git diff` do deploy para ver exatamente o que mudou.
2. Rastreia a causa pelo código — por exemplo, uma variável de ambiente renomeada no código, mas não na configuração.
3. Propõe a correção e mostra o diff; você aprova.
4. Roda os testes e confirma que passam.
5. Faz o commit com mensagem descritiva e, se você pedir, abre o pull request.

Você define o objetivo e aprova as ações. O trabalho braçal fica com ele.

RESPOSTA DO CHAT

Uma hipótese bem fundamentada — que você ainda precisa aplicar e testar.

RESPOSTA DO CLAUDE CODE

Uma correção já aplicada e testada no seu ambiente real.

Pedidos que funcionam muito bem no Claude Code

"Escreva testes para o módulo de autenticação, rode e corrija as falhas."

"Analise os logs do nginx da última hora e me diga de onde vêm os erros 502."

"Este script de backup em PowerShell parou de funcionar ontem. Descubra o porquê."

"Atualize as dependências, rode a suíte de testes e me mostre o que quebrou."

"Explique a arquitetura deste repositório como se eu tivesse acabado de entrar no time."

"Crie um Dockerfile e um docker-compose para este serviço e valide o build."

E quando o chat ainda é a melhor escolha?

- Perguntas rápidas e conceituais ("RAID 5 ou 10?")
- Escrever e revisar textos, e-mails e propostas
- Analisar um PDF ou uma planilha avulsa
- Pesquisa e brainstorming

Regra prática: *se a resposta vai virar arquivo, comando ou commit, é trabalho para o Claude Code.*

PARTE 2

Por que o Claude Code rende mais

Dez argumentos práticos para quem trabalha com TI — do fim do copiar-e-colar à automação no CI.

01

Fim do copiar-e-colar

No chat, o Claude só enxerga o que você cola. Aqui ele lê o projeto, roda os comandos e lê a saída sozinho. O tempo gasto transportando contexto vira tempo de decisão.

02

Ele confere o próprio trabalho

Roda os testes, lê o erro, corrige e roda de novo até a tarefa fechar. A resposta deixa de ser um palpite e vira um resultado testado.

03

Contexto real, não fragmentos

Entende como os arquivos se conectam e edita vários de uma vez. Com um plugin de LSP, vê erros de tipo logo após editar e navega por definições, como a sua IDE.

04

Fala a língua do terminal

Segue a filosofia Unix: entra em pipes, scripts, cron e CI.

```
$ tail -200 app.log |
    claude -p "aponte anomalias nestes logs"
$ git diff main --name-only |
    claude -p "procure falhas de segurança"
```

05

Vai muito além de programar

Se dá para fazer pela linha de comando, ele também faz: scripts Bash e PowerShell, análise de logs, Docker, configurações, documentação, migrações, auditoria de dependências.

06

Memória de projeto versionada

O `CLAUDE.md` guarda padrões, comandos de build e regras do tipo “nunca faça X”. É lido em toda sessão e vai para o git: o time inteiro compartilha o mesmo contexto.

07

Conecta com as suas ferramentas

Via MCP, o padrão aberto de integração, consulta o banco de dados, atualiza tickets no Jira, lê o Google Drive e o Slack ou usa a ferramenta interna da empresa.

08

Automação de verdade

Hooks garantem ações em todo evento (formatar após editar, bloquear comandos perigosos). No CI, revisa PRs via GitHub Actions ou GitLab. **Rotinas** agendadas rodam até com o PC desligado.

09

Trabalho em paralelo

Subagentes pesquisam em contexto isolado e devolvem só o resumo. Rode várias sessões ao mesmo tempo, comece uma tarefa no celular e continue no terminal depois.

10

Sem custo extra

Incluído nos planos Pro, Max, Team e Enterprise. Quem já assina o Pro (US$ 20/mês, ou US$ 17/mês no anual) já tem — no app desktop, é só abrir a aba **Code**.

PARTE 2 · SEGURANÇA

“Deixar uma IA rodar comandos na minha máquina?”

É a primeira pergunta de qualquer profissional de TI — e a resposta está no desenho da ferramenta. Você escolhe quanta autonomia dar, e as regras são aplicadas pelo próprio Claude Code, não pelo modelo: não dependem da “boa vontade” da IA.

MODO	O QUE RODA SEM PERGUNTAR	QUANDO USAR
Manual	Só leituras. Ele pede permissão antes de editar arquivos, rodar comandos ou acessar a rede	Trabalho sensível, código desconhecido
Accept edits	Leituras, edições e comandos simples de arquivos (`mkdir`, `mv`, `cp`)	Iterar em código que você está revisando
Plan	Só explora e propõe um plano; não edita seus arquivos até você aprovar	Entender o problema antes de mexer
Auto	Tudo, com um classificador — um segundo modelo — revisando as ações em segundo plano e bloqueando as arriscadas	Tarefas longas, sem aprovar cada passo

`Shift+Tab` alterna entre os modos. Nas versões recentes, as sessões no terminal começam no Auto; quem prefere aprovar tudo usa o Manual. Para CI existe o `dontAsk` (só roda o que estiver pré-aprovado); o `bypassPermissions`, que pula as checagens, é só para contêineres e VMs isolados.

Regras allow / ask / deny

Por ferramenta e por comando, versionadas em `.claude/settings.json`. Regras de *deny* valem em todos os modos.

```
{
  "permissions": {
    "allow": ["Bash(npm run *)",
              "Bash(git commit *)"],
    "deny":  ["Bash(git push *)",
              "Read(./.env)"]
  }
}
```

Sandbox no sistema operacional

Restringe o acesso dos comandos de shell a arquivos e à rede no nível do SO — e continua valendo mesmo que uma injeção de prompt engane o modelo.

Checkpoints

Antes de editar um arquivo, ele salva o estado anterior; `Esc` duas vezes volta atrás. Ações em sistemas remotos (bancos, APIs, deploys) não têm checkpoint: essas você controla pelas permissões.

Travas que nenhum modo ignora

Remoções com `rm` / `rmdir` em caminhos críticos e ações cobertas por uma regra de *ask* nunca são aprovadas automaticamente — em nenhum modo, nem no `bypassPermissions`.

Limites ditos na conversa

Se você disser “não faça push” ou “espere minha revisão antes do deploy”, o classificador do modo Auto bloqueia as ações correspondentes até você liberar.

Políticas da empresa

Configurações gerenciadas (*managed settings*) que o usuário não sobrescreve: desativar modos, liberar ou bloquear marketplaces de plugins e forçar a instalação dos aprovados.

Histórico auditável

Cada mensagem, ação e resultado fica salvo localmente em texto puro (JSONL, em `~/.claude/projects/`): dá para auditar, retomar e bifurcar sessões.

PARTE 3

Plugins: o gerenciador de pacotes

Um plugin é uma pasta de componentes — skills, subagentes, hooks, servidores MCP e outros — que o Claude Code instala e carrega como uma unidade. Para quem é de TI, a analogia é imediata: é o **apt**, o **npm** ou o **brew** das capacidades do Claude.

Marketplace ≈ repositório de pacotes	Um catálogo — um arquivo `marketplace.json` num repositório git — que lista os plugins e de onde baixar cada um. O oficial da Anthropic já vem configurado.
Plugin ≈ pacote	Uma pasta com manifesto (`.claude-plugin/plugin.json`): nome, versão, descrição e os componentes.
/plugin install ≈ apt install	Instala com um comando, como `/plugin install commit-commands@claude-plugins-official`, e recebe atualizações do marketplace.
Escopos ≈ global × por projeto	**User:** em todos os seus projetos. **Project:** para todo mundo que trabalha no repositório. **Local:** só para você, só naquele repositório.

Anatomia de um plugin

`meu-plugin/`	A pasta inteira é instalada, atualizada e carregada como uma unidade.
`├── .claude-plugin/plugin.json`	**Manifesto** — nome, versão e descrição do plugin.
`├── skills/review/SKILL.md`	**Skill** — instruções que o Claude carrega quando são relevantes; aqui, vira o comando `/meu-plugin:review`.
`├── agents/security-reviewer.md`	**Subagente** — especialista com contexto próprio, para quem o Claude delega tarefas.
`├── hooks/hooks.json`	**Hooks** — ações garantidas em eventos: rodar o linter após cada edição, bloquear arquivos protegidos.
`├── .mcp.json`	**Servidores MCP** — ferramentas de sistemas externos: banco de dados, Jira, Slack.
`├── .lsp.json`	**Servidores LSP** — erros de tipo e navegação por símbolo, como na IDE.
`├── monitors/monitors.json`	**Monitors** — comandos em segundo plano (como `tail -F error.log`) que avisam o Claude.
`└── bin/`	**Executáveis** — scripts no `PATH` enquanto o plugin está ativo, como o CLI interno da empresa.

Nenhum componente é obrigatório: cada plugin usa só o que precisa. Também pode trazer estilos de resposta (*output styles*), temas de cores e configurações padrão.

PARTE 3 · VANTAGENS

As vantagens dos plugins

O que transforma peças soltas em algo instalável, versionado e compartilhável.

01

Setup completo em um comando

Em vez de configurar skills, hooks e integrações um a um, você instala um pacote pronto e testado — que se atualiza sozinho pelo marketplace oficial.

02

O time inteiro no mesmo padrão

No escopo de projeto, o plugin fica no `.claude/settings.json` do repositório: cada pessoa instala uma vez e o time inteiro trabalha com o mesmo fluxo, hooks e integrações.

03

Modular e transparente

Desative sem desinstalar, veja o que está parado e, antes de instalar, quantos tokens o plugin adiciona a cada mensagem. Comandos em namespace evitam conflitos.

04

Reaproveitamento entre projetos

Empacote o setup de um repositório e leve para os outros — ou publique no marketplace da empresa, em qualquer repositório git (GitHub, GitLab, servidor interno).

05

Governança corporativa

Por configurações gerenciadas, a empresa libera ou bloqueia marketplaces e força a instalação dos plugins aprovados.

06

Ecossistema em crescimento

Marketplace oficial (Anthropic e parceiros como GitHub, Atlassian, Slack, Figma, Vercel e Supabase), marketplace da comunidade e de terceiros.

O mesmo plugin rende mais no Claude Code

O formato de plugin também funciona no chat do claude.ai — mas lá só uma parte dos componentes é carregada:

Componente	Chat (claude.ai)	Claude Code
Skills	✅ Carrega	✅ Carrega
Comandos	🌓 Vira skill; o Claude aplica quando faz sentido	✅ Carrega
Subagentes	❌ Ignorados	✅ Carrega
Hooks	❌ Ignorados	✅ Carrega
Servidor MCP remoto	🌓 Pela aba Conectores, depois de conectar	✅ Carrega
Servidor MCP local	❌ Ignorado	✅ Carrega
Executáveis (`bin/`)	❌ O plugin nem instala	✅ Carrega
LSP, estilos e temas	❌ Ignorados	✅ Carrega

Fonte: tabela de compatibilidade de plugins da documentação oficial (claude.com/docs/plugins/platform-support).

PARTE 3 · NA PRÁTICA

Por onde começar: plugins que valem a instalação

Seleção do marketplace oficial. Os de fluxo têm o selo “Anthropic verified”.

FLUXO DE TRABALHO

Commit Commands — commit, push e criação de pull request

Code Review — revisão com agentes especializados e filtro por nível de confiança

PR Review Toolkit — revisa comentários, testes, tratamento de erros, tipos e qualidade

Feature Dev — desenvolve features com agentes de exploração, design e revisão

Security Guidance — hook que alerta sobre injeção de comandos, XSS e padrões inseguros

Code Simplifier — simplifica e refina o código alterado recentemente

CLAUDE.md Management — audita o CLAUDE.md e registra aprendizados

Claude Code Setup — analisa o projeto e recomenda automações sob medida

INTEGRAÇÕES

GitHub — servidor MCP oficial para gerenciar repositórios

Atlassian — Jira e Confluence: issues, documentação e sprints

Slack — servidor MCP oficial do Slack

Playwright — automação de navegador e testes end-to-end (Microsoft)

Context7 — documentação atualizada de bibliotecas, consultada na hora

Supabase — banco, autenticação e storage; roda SQL

Chrome DevTools — controla e inspeciona um Chrome em execução

Figma — lê arquivos de design e traduz componentes em código

INTELIGÊNCIA DE CÓDIGO (LSP)

`pyright-lsp` Python · `typescript-lsp` TS/JS · `jdtls-lsp` Java · `csharp-lsp` C# · `gopls-lsp` Go · `rust-analyzer-lsp` Rust · `php-lsp` PHP · `clangd-lsp` C/C++ · `kotlin-lsp` Kotlin · `ruby-lsp` Ruby · `swift-lsp` Swift · `lua-lsp` Lua

Cada um precisa do language server instalado na máquina — por exemplo, `npm install -g typescript-language-server typescript` para TypeScript.

INSTALAR E GERENCIAR

```
> /plugin                                          # painel: Discover, Installed, Marketplaces
> /plugin install commit-commands@claude-plugins-official
> /plugin marketplace add sua-empresa/plugins      # marketplace do time (repo git)
$ claude plugin list                               # no shell: instalados e escopos
$ claude plugin disable nome@marketplace           # desliga sem desinstalar
```

⚠ **Plugin é código.** Ele roda com os seus privilégios de usuário. Antes de instalar, o painel mostra tudo o que ele adiciona (comandos, agentes, hooks, servidores MCP e LSP). Os marketplaces oficiais são da Anthropic; marketplaces de terceiros não passam pela revisão dela — instale só de fontes em que você confia.

MÃOS À OBRA

Como começar em 10 minutos

Da instalação ao primeiro plugin, sem pagar nada além do Pro ou do Max.

1 Instale

```
# macOS, Linux e WSL
$ curl -fsSL https://claude.ai/install.sh | bash
# Windows (PowerShell)
PS> irm https://claude.ai/install.ps1 | iex
```

Alternativas: `brew install --cask claude-code` ou `winget install Anthropic.ClaudeCode`.

2 Abra um projeto e inicie

Entre na pasta do projeto, rode `claude` e faça login com a mesma conta do Claude (Pro ou superior).

3 Gere a memória do projeto

Rode `/init`: ele analisa o repositório e cria um `CLAUDE.md` inicial, que você ajusta com os padrões do time.

4 Peça algo real

"Explique a arquitetura deste projeto" ou "escreva testes para o módulo X, rode e corrija as falhas".

5 Instale os primeiros plugins

`/plugin` → aba **Discover** → `commit-commands` e o LSP da sua linguagem.

Prefere interface gráfica? Aba **Code** do app desktop, extensões para VS Code e JetBrains, ou claude.ai/code.

Cola rápida

`Shift+Tab`	alterna os modos de permissão	`/plugin`	instala e gerencia plugins
`Esc`	interrompe o que ele está fazendo	`/mcp`	mostra as integrações conectadas
`Esc Esc`	volta a um checkpoint anterior	`/model`	troca o modelo
`/init`	gera o CLAUDE.md do projeto	`/context`	mostra o que ocupa o contexto
`claude --continue`	retoma a última sessão	`/compact`	resume uma conversa longa
`claude -p "…"`	modo não interativo, para scripts	`/doctor`	diagnostica a instalação

O chat é um consultor brilhante ao telefone. O Claude Code é esse mesmo consultor sentado ao seu lado, com acesso ao terminal — e você decide até onde ele pode ir.

Fontes (consultadas em outubro de 2026): code.claude.com/docs — Overview, How Claude Code works, Extend Claude Code, Permissions, Permission modes, Plugins, Install plugins, Plugin components, Anthropic's marketplaces, Code intelligence; claude.com/docs/plugins/platform-support; claude.com/pricing; claude.com/marketplace/plugins; support.claude.com ("Create and edit files with Claude"). Recursos, planos e preços mudam com frequência — confira sempre a documentação oficial.